Let’s complete the business logic for Catalog Service by implementing the two exceptions we used in listing 3.5.

`BookAlreadyExistsException` is a runtime exception thrown when we try to add a book to the catalog that is already there. It prevents duplicate entries in the catalog.

**Listing 3.8 Exception thrown when adding a book that already exists**

```
package com.polarbookshop.catalogservice.domain;

public class BookAlreadyExistsException extends RuntimeException {
  public BookAlreadyExistsException(String isbn) {
    super("A book with ISBN " + isbn + " already exists.");
  }
}
```

`BookNotFoundException` is a runtime exception thrown when we try to fetch a book that is not in the catalog.

**Listing 3.9 Exception thrown when a book cannot be found**

```
package com.polarbookshop.catalogservice.domain;

public class BookNotFoundException extends RuntimeException {
  public BookNotFoundException(String isbn) {
    super("The book with ISBN " + isbn + " was not found.");
  }
}
```

That completes the business logic for the Catalog Service. It’s relatively simple, but it’s recommended not to be influenced by how data is persisted or exchanged with clients. The business logic should be independent of anything else, including the API. If you’re interested in this topic, I suggest exploring the concepts of *domain-driven design* and *hexagonal architecture*.

## *3.3.2 Implementing a REST API with Spring MVC*